

# C++ Set 라이브러리 사용법

중복 없는 정렬 집합 set부터 빠른 검색을 위한 unordered\_set까지, 함수·예제·비교표를 한 번에 정리한 학습 자료

핵심 한 줄                      set은 자동 정렬, unordered\_set은 평균적으로 더 빠른 검색에 강합니다.

set은 같은 값을 중복해서 저장하지 않고, 자동으로 정렬해 주는 자료구조입니다.

C언어의 배열과 비교하면 다음과 같습니다.

| 구분     | C 배열       | C++ set     |
|--------|------------|-------------|
| 중복 값   | 저장 가능      | 저장 불가       |
| 정렬     | 직접 해야 함    | 자동 정렬       |
| 크기     | 보통 미리 지정   | 자동으로 늘어남    |
| 값 추가   | 인덱스에 직접 저장 | insert() 사용 |
| 값 검색   | 반복문으로 검색   | find() 사용   |
| 인덱스 접근 | arr[0] 가능  | s[0] 불가능    |

## 1. set 기본 사용법

set을 사용하려면 <set>을 포함해야 합니다.

```
#include <iostream>
#include <set>      // set을 사용하기 위한 헤더
using namespace std;

int main()
{
    set<int> numbers;    // int 값을 저장하는 set 생성

    numbers.insert(30);  // 30 추가
    numbers.insert(10);  // 10 추가
    numbers.insert(20);  // 20 추가
    numbers.insert(10);  // 중복된 10은 추가되지 않음

    for (int number : numbers) // set의 값을 처음부터 순서대로 꺼냄
    {
        cout << number << " "; // 10 20 30 출력
    }
}
```

출력:

10 20 30

값을 30 → 10 → 20 순서로 넣었지만, set은 자동으로 오름차순 정렬합니다.

## 값의 변화

| 실행 코드      | set에 저장된 값   |
|------------|--------------|
| insert(30) | {30}         |
| insert(10) | {10, 30}     |
| insert(20) | {10, 20, 30} |
| insert(10) | {10, 20, 30} |

마지막 10은 이미 존재하기 때문에 추가되지 않습니다.

## 2. set의 주요 함수

| 함수                 | 역할            | 사용 예시              |
|--------------------|---------------|--------------------|
| insert(value)      | 값 추가          | s.insert(10);      |
| erase(value)       | 해당 값 삭제       | s.erase(10);       |
| find(value)        | 값 검색          | s.find(10);        |
| count(value)       | 값이 있는지 확인     | s.count(10);       |
| size()             | 저장된 값의 개수     | s.size();          |
| empty()            | 비어 있는지 확인     | s.empty();         |
| clear()            | 모든 값 삭제       | s.clear();         |
| begin()            | 첫 번째 값의 위치    | s.begin();         |
| end()              | 마지막 다음 위치     | s.end();           |
| lower_bound(value) | value 이상인 첫 값 | s.lower_bound(20); |
| upper_bound(value) | value보다 큰 첫 값 | s.upper_bound(20); |

## 3. insert()로 값 추가하기

```
set<int> numbers;    // 빈 set 생성

numbers.insert(10);   // 10 추가
numbers.insert(20);   // 20 추가
numbers.insert(10);   // 중복이므로 추가되지 않음
```

insert()는 값이 실제로 들어갔는지도 확인할 수 있습니다.

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> numbers; // 정수를 저장하는 set

    auto result = numbers.insert(10); // 10 추가 결과를 저장

    if (result.second) // 추가에 성공했다면 true
    {
        cout << "추가 성공\n";
    }
    else
    {
        cout << "이미 존재하는 값\n";
    }

    result = numbers.insert(10); // 같은 값 다시 추가

    if (!result.second) // 추가에 실패했다면
    {
        cout << "10은 이미 있습니다.\n";
    }
}
```

insert()의 반환 결과에는 두 가지 정보가 들어 있습니다.

| 값             | 의미                  |
|---------------|---------------------|
| result.first  | 해당 값이 있는 위치         |
| result.second | 추가 성공 여부 true/false |

## 4. find()로 값 검색하기

find()는 값이 존재하면 그 값의 위치를 반환하고, 존재하지 않으면 end()를 반환합니다.

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> numbers = {10, 20, 30}; // 초기값을 가진 set

    auto it = numbers.find(20); // 20을 검색

    if (it != numbers.end()) // end가 아니면 값을 찾은 것
    {
        cout << "찾은 값: " << *it; // *it로 실제 값 출력
    }
    else
    {
        cout << "값이 없습니다.";
    }
}
```

여기서 it는 반복자(iterator)입니다.

처음에는 어렵게 생각하지 말고, set 안의 특정 값을 가리키는 표시라고 이해하면 됩니다.

set: {10} {20} {30} ↑ it

\*it라고 쓰면 it가 가리키는 실제 값인 20을 얻습니다.

## 5. count()로 존재 여부 확인하기

set은 중복을 허용하지 않으므로 count() 결과는 항상 0 또는 1입니다.

```
set<int> numbers = {10, 20, 30}; // 값 저장

cout << numbers.count(20);      // 존재하므로 1
cout << numbers.count(50);      // 존재하지 않으므로 0
```

따라서 간단하게 존재 여부만 확인할 때는 다음처럼 사용할 수 있습니다.

```
if (numbers.count(20) == 1)      // 20이 존재한다면
{
    cout << "20이 있습니다.";
}

C++20 이상에서는 contains()도 사용할 수 있습니다.

C++
if (numbers.contains(20))        // 20이 존재하면 true
{
    cout << "20이 있습니다.";
}
```

초보 단계에서는 다음처럼 구분하면 됩니다.

| 함수         | 사용 목적                         |
|------------|-------------------------------|
| find()     | 값을 찾고 그 위치까지 사용하고 싶을 때        |
| count()    | 값이 있는지만 확인할 때                 |
| contains() | 값이 있는지만 직관적으로 확인할 때, C++20 이상 |

## 6. erase()로 값 삭제하기

값을 직접 지정해서 삭제

```
set<int> numbers = {10, 20, 30}; // 초기값

numbers.erase(20);              // 값 20 삭제
```

결과:

{10, 30}

없는 값을 삭제해도 오류는 발생하지 않습니다.

```
numbers.erase(100); // 100이 없으므로 아무 일도 일어나지 않음
```

## 반복자를 사용해서 삭제

```
auto it = numbers.find(20);    // 20의 위치 검색

if (it != numbers.end())      // 20을 찾았다면
{
    numbers.erase(it);        // 해당 위치의 값 삭제
}
7. size(), empty(), clear()
C++
set<int> numbers = {10, 20, 30}; // 값 3개 저장

cout << numbers.size() << " \n"; // 저장된 값 개수: 3

if (!numbers.empty())         // 비어 있지 않다면
{
    cout << "값이 있습니다. \n";
}

numbers.clear();              // 모든 값 삭제

cout << numbers.size();       // 0
```

| 함수      | 반환 결과             |
|---------|-------------------|
| size()  | 값의 개수             |
| empty() | 비어 있으면 true       |
| clear() | 모든 값을 삭제하며 반환값 없음 |

## 8. set 전체 값 출력하기

### 범위 기반 for문 사용

가장 간단한 방법입니다.

```
set<int> numbers = {30, 10, 20}; // 자동으로 10, 20, 30 정렬

for (int number : numbers)      // set의 값을 하나씩 number에 복사
{
    cout << number << " ";      // 10 20 30 출력
}
```

복사를 피하고 싶다면 참조자를 사용할 수 있습니다.

```
for (const int& number : numbers) // 값을 복사하지 않고 읽기만 함
{
    cout << number << " ";
}
```

const가 붙은 이유는 set 안의 값을 직접 수정하면 정렬 순서가 깨질 수 있기 때문입니다.

### 반복자 사용

```
for (auto it = numbers.begin(); it != numbers.end(); ++it)
{
    cout << *it << " "; // it가 가리키는 값 출력
}
```

## 9. set은 인덱스를 사용할 수 없다

다음 코드는 불가능합니다.

```
set<int> numbers = {10, 20, 30};

cout << numbers[0]; // 오류: set은 []를 지원하지 않음
```

set은 배열처럼 값이 일렬로 연속 저장된 구조가 아니기 때문입니다.

첫 번째 값을 얻고 싶다면 begin()을 사용합니다.

```
cout << *numbers.begin(); // 가장 작은 값인 10 출력
```

마지막 값을 얻으려면 다음처럼 사용합니다.

```
cout << *numbers.rbegin(); // 가장 큰 값인 30 출력
```

| 표현          | 의미      |
|-------------|---------|
| *s.begin()  | 가장 작은 값 |
| *s.rbegin() | 가장 큰 값  |

단, set이 비어 있을 때 사용하면 안 됩니다.

```
if (!numbers.empty()) // 값이 있을 때만
{
    cout << *numbers.begin(); // 가장 작은 값 출력
}
```

## 10. 내림차순으로 정렬하기

기본 set은 오름차순입니다.

```
set<int> numbers = {30, 10, 20};
```

저장 순서:

10 20 30

내림차순으로 저장하려면 greater<int>를 사용합니다.

```
#include <iostream>
#include <set>
#include <functional>      // greater를 위한 헤더
using namespace std;

int main()
{
    set<int, greater<int>> numbers; // 내림차순 set

    numbers.insert(10);          // 값 추가
    numbers.insert(30);
    numbers.insert(20);

    for (int number : numbers)
    {
        cout << number << " ";    // 30 20 10
    }
}
```

| 선언                        | 정렬 방식 |
|---------------------------|-------|
| set<int> s;               | 오름차순  |
| set<int, less<int>> s;    | 오름차순  |
| set<int, greater<int>> s; | 내림차순  |

## 11. 문자열을 저장하는 set

```
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main()
{
    set<string> books;          // 문자열을 저장하는 set

    books.insert("Harry Potter"); // 책 이름 추가
    books.insert("C Programming");
    books.insert("Animal Farm");
    books.insert("Harry Potter"); // 중복이므로 추가되지 않음

    for (const string& book : books) // 문자열을 복사하지 않고 읽음
    {
        cout << book << '\n';
    }
}
```

문자열은 사전순으로 정렬됩니다.

Animal Farm C Programming Harry Potter

대문자와 소문자는 서로 다른 값으로 취급합니다.

```
books.insert("apple");
books.insert("Apple");
```

두 문자열은 서로 다르므로 둘 다 저장됩니다.

## 12. lower\_bound()와 upper\_bound()

다음 set이 있다고 해보겠습니다.

{10, 20, 30, 40, 50}

**lower\_bound(value)**

value 이상인 첫 번째 값을 찾습니다.

```
auto it = numbers.lower_bound(25); // 25 이상인 첫 값
cout << *it;                      // 30
upper_bound(value)
```

value보다 큰 첫 번째 값을 찾습니다.

```
auto it = numbers.upper_bound(30); // 30보다 큰 첫 값
cout << *it;                      // 40
```

| 코드              | 찾는 조건 | 결과 |
|-----------------|-------|----|
| lower_bound(30) | 30 이상 | 30 |
| upper_bound(30) | 30 초과 | 40 |
| lower_bound(25) | 25 이상 | 30 |
| upper_bound(25) | 25 초과 | 30 |

결과가 없으면 end()가 반환됩니다.

```
auto it = numbers.lower_bound(100); // 100 이상인 값 검색

if (it == numbers.end())           // 해당 값이 없다면
{
    cout << "조건에 맞는 값이 없습니다.";
}
```

## 13. unordered\_set은 무엇인가?

unordered\_set도 set처럼 중복 값을 저장하지 않습니다.

하지만 가장 큰 차이는 자동 정렬을 하지 않는다는 것입니다.



```
#include <iostream>
#include <unordered_set> // unordered_set 헤더
using namespace std;

int main()
{
    unordered_set<int> numbers; // 정렬하지 않는 집합

    numbers.insert(30); // 값 추가
    numbers.insert(10);
    numbers.insert(20);
    numbers.insert(10); // 중복 값은 저장되지 않음

    for (int number : numbers)
    {
        cout << number << " "; // 출력 순서는 보장되지 않음
    }
}
```

출력은 다음 중 하나처럼 나올 수 있습니다.

20 10 30

또는 실행 환경에 따라:

30 20 10

중요한 점은 출력 순서를 예상하면 안 된다는 것입니다.

## 14. set과 unordered\_set 비교

| 구분            | set         | unordered_set   |
|---------------|-------------|-----------------|
| 중복 값          | 허용하지 않음     | 허용하지 않음         |
| 값의 순서         | 자동 정렬       | 순서 없음           |
| 기본 정렬         | 오름차순        | 해당 없음           |
| 내부 방식         | 정렬된 트리 구조   | 해시 구조           |
| 평균 검색 속도      | $O(\log N)$ | $O(1)$          |
| 최악 검색 속도      | $O(\log N)$ | $O(N)$          |
| 최솟값·최댓값       | 쉽게 구할 수 있음  | 바로 구하기 어려움      |
| lower_bound() | 사용 가능       | 사용 불가           |
| upper_bound() | 사용 가능       | 사용 불가           |
| 사용할 헤더        | <set>       | <unordered_set> |

여기서 시간 복잡도를 쉽게 말하면 다음과 같습니다.

set: 값이 많아져도 검색 속도가 안정적입니다.

unordered\_set: 보통은 set보다 검색이 빠릅니다.

unordered\_set: 대신 값의 순서를 보장하지 않습니다.

## 15. unordered\_set이 값을 찾는 방식

unordered\_set은 값이 들어오면 해시 함수를 이용해 저장 위치를 결정합니다.

예를 들어 실제 계산은 구현마다 다르지만, 이해를 위해 다음처럼 생각할 수 있습니다.

저장 위치 = 값 % 저장 공간의 개수

| 값  | 계산 예시  | 저장 위치 |
|----|--------|-------|
| 10 | 10 % 5 | 0번    |
| 12 | 12 % 5 | 2번    |
| 18 | 18 % 5 | 3번    |

따라서 값을 처음부터 하나씩 비교하지 않고, 계산한 위치로 바로 이동해 빠르게 찾을 수 있습니다.

**unordered\_set 저장 공간**

0번: 10

1번:

2번: 12 3번: 18

4번:

단, 서로 다른 값이 같은 위치로 계산되는 경우가 있습니다. 이것을 해시 충돌이라고 합니다.

$10 \% 5 = 0$   $20 \% 5 = 0$

이 경우 10과 20을 같은 위치에서 따로 관리해야 하므로 성능이 느려질 수도 있습니다.

## 16. unordered\_set의 주요 함수

대부분 set과 같습니다.

| 함수              | 역할                |
|-----------------|-------------------|
| insert(value)   | 값 추가              |
| erase(value)    | 값 삭제              |
| find(value)     | 값 검색              |
| count(value)    | 값 존재 여부           |
| contains(value) | 값 존재 여부, C++20 이상 |
| size()          | 값 개수              |
| empty()         | 비었는지 확인           |
| clear()         | 전체 삭제             |
| begin()         | 순회 시작 위치          |
| end()           | 순회 종료 위치          |

하지만 다음 함수는 사용할 수 없습니다.

```
unordered_set<int> numbers;

numbers.lower_bound(10); // 오류
numbers.upper_bound(10); // 오류
```

unordered\_set은 정렬되어 있지 않기 때문입니다.

## 17. 언제 무엇을 사용해야 할까?

| 상황                     | 추천 자료구조       |
|------------------------|---------------|
| 중복을 제거하고 정렬도 필요함       | set           |
| 값이 존재하지만 빠르게 확인        | unordered_set |
| 최솟값·최댓값이 필요함           | set           |
| 특정 값 이상 또는 초과 값을 찾아야 함 | set           |
| 출력 순서가 중요하지 않음         | unordered_set |
| 중복 값도 저장해야 함           | multiset      |
| 인덱스로 접근해야 함            | vector        |

예를 들어 학생 번호 중복을 확인할 때는 unordered\_set이 적합합니다.

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
    unordered_set<int> studentIds; // 학생 번호 저장

    int id; // 입력받을 학생 번호
    cin >> id; // 학생 번호 입력

    if (studentIds.count(id) == 1) // 이미 저장된 번호라면
    {
        cout << "중복된 학생 번호입니다.";
    }
    else
    {
        studentIds.insert(id); // 중복이 아니면 추가
        cout << "학생 번호가 등록되었습니다.";
    }
}
```

정렬된 점수를 출력해야 한다면 set이 적합합니다.

```
set<int> scores = {80, 100, 70, 80}; // 80은 한 번만 저장

for (int score : scores)           // 낮은 점수부터 출력
{
    cout << score << " ";         // 70 80 100
}
```

## 18. multiset과의 차이

set은 중복 값을 저장할 수 없지만, multiset은 중복 값을 저장할 수 있습니다.

```
#include <set>
using namespace std;

set<int> s;           // 중복 불가능
multiset<int> ms;     // 중복 가능

s.insert(10);
s.insert(10);         // 하나만 저장됨

ms.insert(10);
ms.insert(10);        // 두 개 모두 저장됨
```

| 자료구조               | 중복  | 정렬      |
|--------------------|-----|---------|
| set                | 불가능 | 자동 정렬   |
| multiset           | 가능  | 자동 정렬   |
| unordered_set      | 불가능 | 정렬하지 않음 |
| unordered_multiset | 가능  | 정렬하지 않음 |

## 실수하기 쉬운 부분

### 1. set에 같은 값을 여러 번 넣어도 하나만 저장된다

```
set<int> numbers;

numbers.insert(10);
numbers.insert(10);
numbers.insert(10);

cout << numbers.size(); // 3이 아니라 1
```

### 2. set은 []로 접근할 수 없다

```
numbers[0]; // 오류
```

### 3. unordered\_set의 출력 순서를 믿으면 안 된다

```
unordered_set<int> numbers = {10, 20, 30};
```

항상 10 20 30으로 출력되는 것이 아닙니다.

## 4. 반복자가 end()인지 확인하지 않고 사용하면 안 된다

```
auto it = numbers.find(100);

cout << *it; // 100이 없다면 잘못된 접근
```

올바른 코드:

```
auto it = numbers.find(100); // 100 검색

if (it != numbers.end())    // 찾은 경우에만
{
    cout << *it;           // 값 사용
}
```

## 5. set 안의 값은 직접 수정할 수 없다

```
set<int> numbers = {10, 20, 30};

auto it = numbers.find(20);
*it = 50; // 오류
```

값을 변경하려면 기존 값을 삭제하고 새 값을 추가해야 합니다.

```
numbers.erase(20); // 기존 값 삭제
numbers.insert(50); // 새로운 값 추가
```

## 핵심 정리표

| 자료구조          | 핵심 특징                | 예시              |
|---------------|----------------------|-----------------|
| vector        | 순서와 중복이 있고 인덱스 사용 가능 | 학생 점수 목록        |
| set           | 중복 없이 자동 정렬          | 정렬된 고유 번호       |
| unordered_set | 중복 없이 빠른 검색, 순서 없음   | 아이디 중복 검사       |
| multiset      | 중복을 허용하면서 자동 정렬      | 같은 점수가 있는 점수 목록 |

한 문장 요약: set은 중복을 제거하면서 자동 정렬하는 자료구조이고, unordered\_set은 정렬을 포기하는 대신 보통 더 빠르게 값을 검색하는 자료구조입니다.

### 출처

ChatGPT 공유 대화 · <https://chatgpt.com/share/6a6838fc-a778-83ee-8729-0c3363173190>  
원문의 설명과 예제 코드를 읽기 좋은 문서 형식으로 재구성했습니다.